C语言指针进阶

作者：张三丰 时间：2024年10月

1. 数组和指针

1.1 指针和数组

指针的本质是地址，而数组的本质是一块内存的地址。

```
//测试代码 main.c
#include <stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,0};
    printf("%zd\n", sizeof(arr));
    printf("%p\n", arr);
    return 0;
}
```

```
40
000000D7A2FFF8A0
```

- 数组的类型是：int[10]，在第一台机器的测试结果中，数组的大小是40个字节。
- 数组的地址是：0x000000D7A2FFF8A0

arr是数组名，是一个特殊的指针，它指向数组的第一个元素，**数组名是数组的首元素的地址**。

例外：

sizeof(arr)是例外，当sizeof作用于数组时，它计算**数组的总内存大小**，而不是指针的大小。特记是例外。

1.2 指针数组

指针数组是一个数组，其元素是指针。

```
//测试代码 main.c
#include <stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,0};
    printf("%zd\n", sizeof(arr));
    printf("%p\n", arr);
    printf("%zd\n", sizeof(arr[0]));
    printf("%p\n", &arr);
    return 0;
}
```

```
40
000000D7A2FFF8A0
8
000000D7A2FFF8A0
```

此时代表的意义分别是：

- arr：数组第一个元素的地址
- arr[0]：数组第一个元素的地址
- &arr：数组的地址

测试：

```
//测试代码 main.c
#include <stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,0};
    printf("%zd\n", sizeof(arr));
    printf("%zd\n", sizeof(arr+0));
    printf("%p\n", *(arr+0));
    printf("%p\n", arr+1);
    printf("%p\n", &arr+1);
    return 0;
}
```

```
40
8
000000D7A2FFF8A0
000000D7A2FFF8A4
000000D7A2FFF8C8
```

思考1：为什么？

```
//测试代码 main.c
#include <stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,0};
    printf("%zd\n", sizeof(arr));
    printf("%zd\n", sizeof(arr+0));
    printf("%p\n", *(arr+0));
    printf("%p\n", arr+1);
    printf("%p\n", &arr+1);
    return 0;
}
```

思考2：数组指针的应用场景

```
//测试代码 main.c
void print_arr(int* ,int ,int);

int main()
{
    int arr[3][5] = {1,2,3,4,5};
    int* p = arr;
    int* pa = &arr[0];
    int(*parr)[5] = &arr[0];
    printf("%p\n", p + 2);
    printf("%p\n", pa + 2);
    print_arr(parr, 3, 5);
    return 0;
}
void print_arr(int* p, int x, int y)
{
    int i = 0;
    int j = 0;
    int* pc = NULL;
    for (i = 0; i < x; i++)
    {
        pc = p + i * y;
        for (j = 0; j < y; j++)
        {
            printf("%d ", *(pc + j));
        }
        printf("\n");
    }
}
```

2. 一维数组传参

数组参数和指针参数在函数调用时的传参方式是相同的。

```
//测试代码 main.c
void test1(int* p, int* pc)
{
    int* pd = sizeof(p+0+sizeof(int));
    printf("%d", *pd);
}

int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,0};
    int* pc = sizeof(arr+sizeof(int));
    printf("%d", *pc);
    test1(arr, pc);
    return 0;
}
```

```
4
8
```

sizeof(arr+0)，是数组指针么？

数组名的两种不同含义中：

int arr = {1,2,3,4,5};

int* p = arr;

数组名单独放在sizeof内部，或者&arr，这时数组名是一个数组的地址，这时数组名是数组。

除此之外，数组名是指向数组首元素的指针。

```
//测试代码 main.c
void test1(int* p, int* pc)
{
    printf("%p", sizeof(p));
}

int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,0};
    int* pc = sizeof(arr+sizeof(int));
    printf("%d", sizeof(pc));
    test1(arr, pc);
    return 0;
}
```

```
000000D7A2FFF8A0
8
000000D7A2FFF8A0
```

结果：

一维数组传参，形参部分都可以写为数组或指针的形式，在形式上写成指针的形式。

3. 二级指针

3.1 概念

二级指针的概念：

指针变量的指针变量

对象的地址指向一个指针的对象。

3.2 表示

```
//测试代码 main.c
int main()
{
    int a = 10;
    int* p = &a;
    int** pp = &p;
    return 0;
}
```

一个数组的地址是二级指针的指针么？

问题2：“a的指针变量”？

```
//测试代码 main.c
int main()
{
    int a = 10;
    int* p = &a;
    int** pp = &p;
    printf("%p", *p);
    return 0;
}
```

```
000000D7A2FFF8A0
```

结果：

0x000000D7A2FFF8A0

问题3：“a的指针的指针”？

```
//测试代码 main.c
int main()
{
    int a = 10;
    int* p = &a;
    int** pp = &p;
    printf("%p", *pp);
    return 0;
}
```

```
000000D7A2FFF8A0
```

结果：

0x000000D7A2FFF8A0

问题4：“a的指针的指针”？

```
//测试代码 main.c
int main()
{
    int a = 10;
    int* p = &a;
    int** pp = &p;
    printf("%p", *pp);
    return 0;
}
```

```
000000D7A2FFF8A0
```

结果：

0x000000D7A2FFF8A0

结果：

二级指针的本质是内存地址，指向一个指针的内存中的内存地址的内容，也就是变量a的地址。

问题5：“a的指针的a”？

```
//测试代码 main.c
int main()
{
    int a = 10;
    int* p = &a;
    int** pp = &p;
    printf("%p", *(*pp) + *p);
    return 0;
}
```

```
000000D7A2FFF8A0
```

结果：

二级指针的本质是内存地址，指向一个指针的内存中的内存地址的内容，也就是变量a的地址。

4. 指针数组

指针数组的概念：

指针数组是存放指针的数组，指针数组的本质是数组，是存放指针的数组。

4.1 定义

指针数组的定义：

int* arr[];

4.2 练习

```
//测试代码 main.c
int main()
{
    int a = 10;
    int b = 20;
    int c = 30;

    int* arr[] = {&a,&b};

    printf("%p", *(arr+0));
    printf("%p", *(arr+1));
    printf("%d", **(arr+0));
    return 0;
}
```

5. 数组指针

数组指针的概念：

数组指针是数组的指针，**数组指针**的本质是指针，数组指针的数组的指针。

5.1 定义

数组指针的定义：

数组指针的定义方式是：int(*)[];

5.2 练习

```
//测试代码 main.c
int main()
{
    int a = 10;
    int b = 20;
    int c = 30;

    int* p = &a;

    int* arr[] = {&a,&b};

    printf("%p", *(arr+0));
    printf("%d", *(*(arr+0)));
    return 0;
}
```

6. 指针习题

6.1 习题1

```
//测试代码 main.c
int main()
{
    int a[5][5] = {1, 2, 3, 4, 5};
    int* ptr1 = (int*)(&a + 1);
    int* ptr2 = (int*)(*(a + 1));
    printf("%d,%d", *(ptr1 - 1), *(ptr2 - 1));
    return 0;
}
```

```
5,2
```

6.2 习题2

```
//测试代码 main.c
int main()
{
    int a[5] = {1, 2, 3, 4, 5};
    int* ptr = (int*)(&a + 1);
    printf("%d,%d", *(a + 1), *(ptr - 1));
    return 0;
}
```

```
2,5
```

a数组

1	2	3	4	5

6.3 习题3

```
//测试代码 main.c
int main()
{
    int a[5][5] = {1,2,3,4,5,6,7,8,9,10};
    int *p = (int *)((int *)a + 1);
    int *q = (int *)*(a + 1);
    printf("%d,%d", *p, *q);
    return 0;
}
```

```
2,6
```

a数组

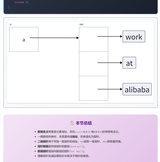
1	2	3	4	5
6	7	8	9	10

6.4 习题4

```
//测试代码 main.c
int main()
{
    int a[] = {1,2,3,4};
    printf("%zd", sizeof(a));
    printf("%zd", sizeof(a+0));
    printf("%zd", sizeof(*a));
    printf("%zd", sizeof(a+1));
    printf("%zd", sizeof(a[1]));
    printf("%zd", sizeof(&a));
    printf("%zd", sizeof(*&a));
    printf("%zd", sizeof(&a+1));
    printf("%zd", sizeof(&a[0]));
    return 0;
}
```

6.5 习题5

```
//测试代码 main.c
int main()
{
    char *a[] = {"work","at","alibaba"};
    char**pp = a;
    pp++;
    printf("%s", *pp);
    return 0;
}
```



🌸 作者感言

- **最最最最最**的指针进阶，我能一口一口吃了吗？小心撑死你的肠胃哦。
- 一个指针的指针，到底是什么概念，你能搞清楚吗？
- 二级指针可不是一个指针的指针的事，一级指针一级指针，一级指针数组，
- 二级指针的本质是内存地址，
- 数组指针的本质是指针，
- 指针的习题是最后的内容，也是大学的考试题。